CSA0609 Design and Analysis of Algorithms

ASSIGNMENT 1 (CO1)

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Total Marks: 50

Algorithms studied: Linear Search (iterative),Binary Search (recursive, on sorted data), Hash-Table Search
Experiments: n = 1,000 -> 10,000 -> 100,000 -> 1,000,000 records

Answer map

Task	Topic
1	Algorithm design (3 strategies, pseudocode, preprocessing, justification)
2	Basic operation
3	Mathematical time-complexity derivation
4	Best / average / worst case
5	Asymptotic analysis
6	Recurrence formulation (recursive binary search)
7	Recurrence solution
8	Implementation and testing
9	Experimental performance data, tables, graphs
10	Theory vs experiment
11	Final design recommendation
12	Day-4 challenge: 50,000 searches/hour, daily updates (A-D, 2 marks each)

0. Results at a glance

Notation: n = number of records; successful search = key present; q = number of searches; alpha = n/m = load factor (m = hash buckets).

Property	A. Linear search	B. Binary search (recursive)	C. Hash table (chaining)
Design strategy	Brute force	Decrease-and-conquer (divide by 2)	Space-time trade-off (hashing)
Style	Iterative	Recursive	Iterative (table lookup)
Preprocessing	None (0)	Sort: Theta(n log n)	Build table: Theta(n) expected
Extra memory	O(1)	O(1) search (O(n) for merge sort)	Theta(n) (buckets + pointers)
Basic operation	key comparison A[i] = K	key comparison A[mid] vs K	key comparisons inside one bucket chain
Best case	Theta(1)	Theta(1)	Theta(1)
Average (successful)	(n+1)/2 = Theta(n)	~ 2log2(n) - 3 = Theta(log n)	1 + alpha/2 = Theta(1)
Worst case	n = Theta(n)	2(floor(log2 n)+1) = Theta(log n)	Theta(n) (all keys collide); Theta(1+alpha) average
Measured comps, n = 10^6	467,756	36.9	1.49
Measured time/search, n = 10^6	20.49 ms	6.84 us	2.50 us
Measured preprocessing, n = 10^6	0	3.81 s (own merge sort) / 0.38 s (sorted())	1.50 s
Total for q searches	q * n/2	n log n + q log n	n + q

Recommendation (Task 11): Binary search on a sorted array for the original general-purpose system (guaranteed worst case, small memory, ordered access, preprocessing repaid after ~40 searches).

Decision at Task 12: MODIFY - add a hash index (rebuilt/merged once per day) as the primary lookup path for 50,000 searches/hour; keep the sorted array for ordered/range queries and as fallback.

TASK 1 - Design the Algorithm

Three meaningfully different strategies are selected. They differ in design paradigm, data organisation and preprocessing - not just in loop-vs-recursion. Registration numbers are unique integers; each record is stored as (RegNo, record pointer).

	A. Linear search	B. Binary search	C. Hash-table search
Design strategy	Brute force / exhaustive scan	Decrease-and-conquer (halve the search space)	Space-time trade-off, direct addressing via hashing
Implementation form	ITERATIVE (while/for loop)	RECURSIVE (function calls itself on one half)	ITERATIVE (hash, then scan one short chain)
Required preprocessing	None	Sort records by RegNo (merge sort, Theta(n log n))	Build hash table with m ~ n buckets (Theta(n))
Data structure	Unsorted array	Sorted array	Array of buckets (linked lists)

1.A Linear search (iterative, brute force)

Basic idea: Examine records one by one from the first to the last until the key matches or the records are exhausted.

Preprocessing: None - works directly on the records in arrival order.

Justification: Baseline for comparison; optimal when n is tiny or only a single search is ever performed (no preprocessing to repay); needs no ordering and no extra memory.

```
ALGORITHM LinearSearch(A[0..n-1], K)
// Input : array A of n registration numbers, key K
// Output: index of K in A, or -1 if absent
for i <- 0 to n-1 do
   if A[i] = K then          // basic operation
      return i
return -1
```

1.B Binary search (recursive, decrease-and-conquer)

Basic idea: Keep the records sorted. Compare K with the middle element; if equal stop, if K is smaller recurse into the left half, otherwise into the right half. The problem size halves at every call.

Preprocessing: Sort the array by registration number once (merge sort below, Theta(n log n)). Binary search is invalid on unsorted data.

Justification: It exploits the ordering of registration numbers to discard half of the candidates per comparison, giving logarithmic growth: 1,000,000 records need at most 20 levels. It is also the natural recursive algorithm for the Task 6-7 recurrence.

```
ALGORITHM BinarySearch(A[lo..hi], K)          // recursive; call as BinarySearch(A, 0, n-1, K)
// A is sorted in ascending order
if lo > hi then return -1                  // base case: empty range
mid <- floor((lo + hi) / 2)
if A[mid] = K then return mid              // basic operation
else if K < A[mid] then
      return BinarySearch(A, lo, mid-1, K) // left half
else    return BinarySearch(A, mid+1, hi, K) // right half

ALGORITHM MergeSort(A[0..n-1])             // preprocessing for B
```

```
if n > 1 then
   copy A[0..floor(n/2)-1] to B ;  copy A[floor(n/2)..n-1] to C
   MergeSort(B) ; MergeSort(C) ; Merge(B, C, A)

ALGORITHM Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])
i <- 0 ; j <- 0 ; k <- 0
while i < p and j < q do
   if B[i] <= C[j] then A[k] <- B[i] ; i <- i+1      // basic operation of sorting
   else                 A[k] <- C[j] ; j <- j+1
   k <- k+1
copy remaining elements of B or C into A
```

1.C Hash-table search (hashing with separate chaining)

Basic idea: A hash function h(K) = K mod m maps the registration number straight to one of m buckets; each bucket holds a short linked list of the records that collide. A search computes h(K) and scans only that bucket.

Preprocessing: Create m ~ n buckets (m a prime near n, e.g. 1,000,003 for 10^6 records) and insert each record: Theta(n) expected.

Justification: It is the only approach whose expected search cost does not grow with n. It tests the space-time trade-off (extra Theta(n) memory) and is the strongest candidate for the high-frequency scenario in Task 12.

```
ALGORITHM BuildHash(A[0..n-1], m)            // preprocessing for C
create m empty lists T[0..m-1]
for i <- 0 to n-1 do
   insert (A[i], i) at the end of list T[ A[i] mod m ]

ALGORITHM HashSearch(T, m, K)                // iterative scan of ONE chain
b <- K mod m                                 // O(1) hash computation
for each (key, pos) in T[b] do
   if key = K then return pos                // basic operation
return -1
```

Diversity check: A = unordered scan, B = order-based divide-and-conquer needing sorted data, C = key-to-address transformation needing a table. Iterative (A, C) and recursive (B) forms are both present. Making B iterative would NOT count as a new approach.

TASK 2 - Identify the Basic Operation

Algorithm	Basic operation	Where it occurs	Why appropriate	How its count varies with n
A. Linear	Key comparison A[i] = K	Inside the loop body, once per record visited	Every other step (loop increment, return) is executed at most as often; comparison is the most frequent, costliest step and independent of the machine	Between 1 and n: grows linearly with n (n for absent key / last record)
B. Binary (search)	Key comparison A[mid] with K	Once per recursive call (at the start of each call, plus the < test)	One comparison is performed per call and the calls form the recursion; calls = levels of the halving	<= floor(log2 n)+1 calls: grows logarithmically (n x1000 -> calls only +10)
B. Merge sort (prep.)	Key comparison B[i] <= C[j] in Merge	Inner while loop of Merge	Dominates the data movement (every comparison moves one element)	n log2 n - O(n): between (n/2)log2 n and n log2 n - n + 1
C. Hash	Key comparison with entries in bucket chain	The for-each loop of HashSearch (one comparison per chain entry)	Hash computation K mod m is a single O(1) operation done once; only chain scanning depends on data distribution	Expected 1 + alpha/2 where alpha = n/m. With m ~ n, alpha ~ 1 -> constant, independent of n
C. Hash build (prep.)	Insertion into a bucket list (hash + append)	Loop of BuildHash	Executed exactly once per record	Exactly n insertions: linear in n

Relation to behaviour: in A the loop terminates only when the key is found, so the number of comparisons equals the position of the key. In B each comparison discards half of the remaining range, so the count equals the number of halvings. In C the hash function jumps directly to one bucket, so only the chain length matters, and chains stay short when m ~ n. Counting comparisons is machine-independent and, as Task 9 will show, is exactly what the measured counters record.

TASK 3 - Derive the Time-Complexity Function

Input size: n = number of student records (the length of the array). q = number of searches. Count unit: basic operation of Task 2.

3.A Linear search

The loop body executes once for every index examined. If the key is at position i (1-indexed) the count is C(i) = i; for an absent key it is n:

C_worst(n) = sum_{i=0}^{n-1} 1 = n

Average, assuming a successful search with each of the n positions equally likely (probability 1/n):

C_avg(n) = (1/n) * sum_{i=1}^{n} i = (1/n) * n(n+1)/2 = (n+1)/2

With probability p that the key is present: C_avg = p(n+1)/2 + (1-p)n.

Preprocessing: none, P_A(n) = 0.

Total for q searches: T_A(n, q) = 0 + q (n+1)/2 = Theta(q n).

Numerical check: n = 1,000,000: worst = 1,000,000; average = 500,000.5; measured average over 200 random hits = 467,756.2 (inside random-sample error: sd of the mean = n/sqrt(12*200) ~ 20,400).

3.B Binary search (recursive) with merge-sort preprocessing

Search. Each call does a constant number of comparisons and recurses on at most floor(n/2) elements. After k calls at most n/2^k elements remain. The recursion ends when the range is empty, i.e. when:

n / 2^k < 1 => k > log2 n => number of calls = floor(log2 n) + 1

Our code performs two comparisons per non-final level (A[mid] = K, then K < A[mid]) and one on the final successful level. Worst case (absent key):

C_worst(n) = 2 (floor(log2 n) + 1)

Average (successful search). In the implicit binary tree of the search, level d holds at most 2^(d-1) keys. For n = 2^k - 1 (full tree) the average number of levels visited is:

D_avg = (1/n) * sum_{d=1}^{k} d * 2^(d-1) = ((k-1) 2^k + 1)/n ~ k - 1 = log2(n+1) - 1

A successful search ending at level d costs 2(d-1)+1 = 2d - 1 comparisons, so C_avg(n) ~ 2 (log2 n - 1) - 1 = 2 log2 n - 3.

Preprocessing (merge sort). Merging two sorted halves of total size n needs between n/2 and n-1 comparisons; sorting n elements satisfies S(n) = 2S(n/2) + (n-1), S(1) = 0, whose solution (Task 7) is:

S_worst(n) = n log2 n - n + 1 ; S_best(n) = (n/2) log2 n ; S_avg(n) ~ n log2 n - 1.25 n

Total: T_B(n, q) = S(n) + q*C(n) = Theta(n log n) + Theta(q log n)

Numerical check n = 10^6: S_avg ~ 10^6*19.93 - 1.25*10^6 = 18,681,569 vs measured 18,674,676 comparisons. Search: 2 log2(10^6) - 3 = 36.86 vs measured 36.88; worst 2*20 = 40 vs measured 40.

3.C Hash table with chaining

Assume simple uniform hashing (each key equally likely to land in each of the m buckets), load factor alpha = n/m.

Unsuccessful search scans one whole chain, whose expected length is the load factor:

C_unsucc = alpha = n/m

Successful search. For the i-th inserted key, the expected number of keys inserted after it in the same bucket is (n-i)/m. Averaging over i:

C_succ = (1/n) sum_{i=1}^{n} (1 + (n-i)/m) = 1 + (1/(nm)) * n(n-1)/2 = 1 + alpha/2 - alpha/(2n)

With m ~ n (alpha = 1) this is ~ 1.5 probes, a constant independent of n.

Preprocessing: n insertions of O(1) each: P_C(n) = n (+ O(m) to allocate buckets) = Theta(n).

Total: T_C(n, q) = n + q (1 + alpha/2) = Theta(n + q).

Numerical check: measured probes per successful search: 1.50, 1.61, 1.59, 1.49 for n = 10^3..10^6 (theory 1.50).

3.D Overall cost: preprocessing + searching

Algorithm	Preprocessing P(n)	One search S(n) avg	Total for q searches	n = 10^6, q = 1.2x10^6 (comparison-equivalents)
A. Linear	0	(n+1)/2	q(n+1)/2	6e+11
B. Binary	~ n log2 n - 1.25n	~ 2 log2 n - 3	n log2 n + q(2 log2 n - 3)	6.29e+07
C. Hash	n	1 + alpha/2	n + 1.5q	2.8e+06

Break-even number of searches (comparison count, n = 10^6). Binary search beats linear once q > S(n)/(C_lin - C_bin) = 18,674,676 / (467,756 - 36.9) ~ 40 searches (theory: n log n / (n/2) = 2 log2 n ~ 40). Hash beats linear after n/(C_lin - 1.5) ~ 2 searches in comparison counts.

TASK 4 - Best, Average and Worst-Case Behaviour

4.A Linear search

- **Best case - key is the first record (A[0] = K).** Loop exits after 1 comparison: C_best = 1 = Theta(1). Measured: 1.
- **Average case - key present, any position equally likely.** Expected scan covers half of the array: (n+1)/2. For absent keys with probability 1-p: p(n+1)/2 + (1-p)n. Measured at n = 10^6: 467,756 (theory 500,000).
- **Worst case - key is the last record OR is absent.** Every record is compared: C_worst = n. Measured: 1,000,000 for both cases at n = 10^6.
- **Why:** no information about the location of K is used, so each unsuccessful comparison eliminates exactly one candidate.

4.B Binary search

- **Best case - key sits at the middle of the sorted range (first mid).** 1 comparison (our counter), Theta(1).
- **Average case - key present, uniformly random.** About half the keys lie on the last level, so the typical key is found within one level of the bottom: ~ log2 n - 1 levels, 2log2 n - 3 comparisons (derivation in 3.B). Measured n = 10^6: 36.88 vs theory 36.86.
- **Worst case - key absent, or in a leaf at maximum depth.** All floor(log2 n)+1 levels are visited: 2(floor(log2 n)+1) comparisons. n = 10^6: 40 (measured 40 for an absent key).
- **Why:** the search space shrinks by half per call, so the number of calls is bounded by the height of the implicit tree, floor(log2 n)+1; keys near the root are found early, but 2^(d-1) of the n keys sit at depth d, so most keys are deep.

Preprocessing (merge sort) is insensitive to input order in its asymptotics: best (n/2)log2 n (already sorted input: every merge ends when one half is exhausted), worst n log2 n - n + 1, always Theta(n log n).

4.C Hash table

- **Best case - key is first in its bucket.** 1 probe, Theta(1).
- **Average case - uniform hashing.** 1 + alpha/2 probes for hits, alpha for misses; Theta(1) when alpha is bounded by a constant. Measured: 1.49-1.61 probes.
- **Worst case - all n keys collide into one bucket (poor hash function, or registration numbers that are all multiples of m).** The single chain has length n, so the search degenerates to linear search: Theta(n). Observed longest chain with random keys: 6, 7, 8, 9 for n = 10^3..10^6 (about ln n/ln ln n growth - still tiny).
- **Why:** cost depends on the chain length, which depends on how well the hash function spreads the keys, not on n itself. Choosing m prime and keeping alpha <= 1 makes the worst case practically unobservable; universal hashing makes it provably improbable.

4.D Measured best / average / worst comparison counts

n	Linear	Linear	Linear	Binary	Binary	Hash avg	Hash

	best	avg	worst	avg	worst (miss)		max chain
1,000	1	478.7	1,000	17.12	20	1.50	6
10,000	1	5,198.7	10,000	23.80	28	1.61	7
100,000	1	52,267.7	100,000	30.39	34	1.59	8
1,000,000	1	467,756.2	1,000,000	36.88	40	1.49	9

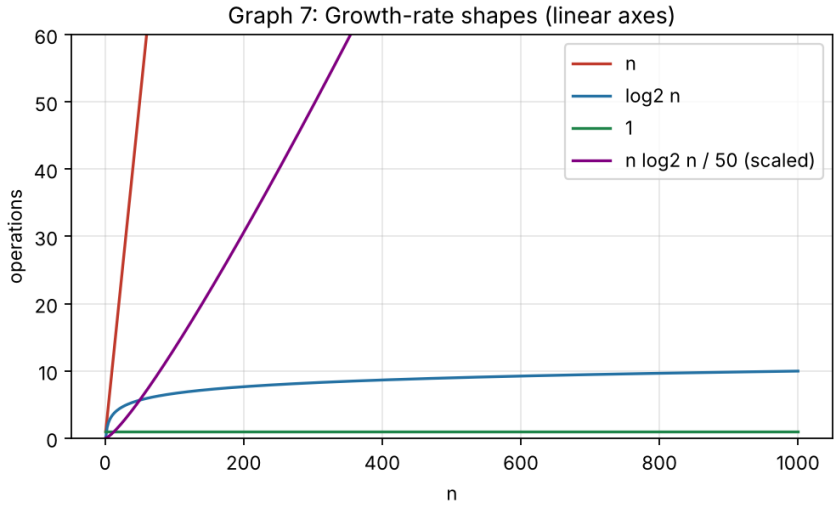
TASK 5 - Asymptotic Analysis

5.A Formal bounds

Definitions used: f(n) = O(g(n)) if f(n) <= c g(n) for all n >= n0; f(n) = Omega(g(n)) if f(n) >= c g(n); f(n) = Theta(g(n)) if both hold.

Algorithm / measure	Exact / derived function	Notation and constants	Meaning
A. Linear - worst	n	O(n) with c = 1, n0 = 1; also Omega(n) => Theta(n)	Tight: exactly linear
A. Linear - best	1	Theta(1) (Omega(1) and O(1))	Lucky first-record hit
A. Linear - average	(n+1)/2	Theta(n): n/2 <= (n+1)/2 <= n for n >= 1	Linear with constant 1/2
B. Binary - worst	2(floor(log2 n)+1)	Theta(log n): log2 n <= C <= 4 log2 n for n >= 2	Logarithmic
B. Binary - best	1	Theta(1)	Hit at first mid
B. Binary - average	2 log2 n - 3	Theta(log n)	Logarithmic
B. Merge sort (prep.)	n log2 n - n + 1 (worst)	Theta(n log n); O(n^2) is true but loose	Super-linear, one-time
C. Hash - average	1 + alpha/2, alpha = n/m <= 1	Theta(1) expected; O(1) with alpha <= c	Independent of n
C. Hash - worst	n	O(n), Omega(n) for the worst input => Theta(n)	Degenerate chain (rare)
C. Hash build	n	Theta(n)	Linear one-time

5.B Growth behaviour as n becomes very large



Graph 7 - Growth-rate shapes. n grows without bound, log2 n flattens, 1 stays constant.

Ordering of growth rates: 1 << log n << n << n log n. Hence for sufficiently large n: Hash (O(1)) < Binary (O(log n)) < Linear (O(n)); the preprocessing costs order as Hash build Theta(n) < Sort Theta(n log n).

5.C Interpretation for 1,000 -> 1,000,000 records (n grows x1000)

Algorithm	Predicted effect of n x 1000	Predicted numbers	Measured (comparisons)	Meaning for the university
Linear	work x1000	500 -> 500,000	479 -> 467,756 (x977)	A 1,000-fold larger database makes every query 1,000x slower: unusable for heavy traffic
Binary	work + 10 levels (x ~2)	2log2n-3: 17 -> 37	17.1 -> 36.9 (x2.15)	1,000x more data costs about twice the work: scales extremely well
Hash	work x1	~1.5 -> ~1.5	1.50 -> 1.49 (x0.99)	Lookup cost independent of database size
Sort (prep.)	x ~ 1,000 * (19.93/9.97) = x2,000	8.7e3 -> 1.87e7	8,712 -> 18,674,676 (x2144)	One-time cost grows slightly faster than the data

Therefore linear search is acceptable only for the 1,000-record (and perhaps 10,000-record) scale; for 10^5-10^6 records its cost per query is dominated by n, while binary and hash searches stay almost flat. The asymptotic behaviour therefore decides practicality when the registration database reaches the million-record scale.

TASK 6 - Formulate the Recurrence Relation

Recursive approach = Algorithm B (recursive binary search). The recurrence is read directly from the code:

```
BinarySearch(A, lo, hi, K):
   if lo > hi then return -1                  <- base case, no further call
   mid <- floor((lo+hi)/2)                    <- constant work
   if A[mid] = K then return mid              <- constant work (stop)
   else if K < A[mid] then return BinarySearch(A, lo, mid-1, K)   <- ONE call
   else                    return BinarySearch(A, mid+1, hi, K)   <- ONE call
```

Component	Identification
Input size of the original problem	n = hi - lo + 1 = number of records in the current range
Size of the recursive subproblem	The left range has floor((n-1)/2) elements, the right range has ceil((n-1)/2) = floor(n/2) elements: at most floor(n/2) in the worst case
Number of recursive calls	Exactly ONE call per invocation (only the half that can contain K is searched; the other half is discarded without being examined) -> a = 1
Work outside the recursive call	Computing mid, comparing K with A[mid] (and the < test): a constant c (2 comparisons in our counter) -> f(n) = Theta(1)
Base case	n = 0 (lo > hi): return -1 immediately, T(0) = constant (0 comparisons)

Combining the components (a = 1 call on size at most n/2, plus constant work c):

$$T(n) = T(\text{floor}(n/2)) + c, \quad n >= 1$$

$$T(0) = d \quad \text{(constant base case)}$$

Comparison-count version (basic operation): each call performs 2 comparisons before recursing:

$$C(n) = C(\text{floor}(n/2)) + 2, \quad n >= 1; \quad C(0) = 0 \quad \text{(worst case, absent key)}$$

Why a worst-case recurrence: the recursion continues to the empty range only when the key is absent or at maximum depth, and the larger half has size floor(n/2) - so the recurrence with equality gives the worst case and an upper bound for all other cases.

Recurrence for the recursive preprocessing step (merge sort)

MergeSort splits n elements into two halves (a = 2 calls, each on n/2 elements) and merges with Theta(n) comparisons:

$$S(n) = 2\,S(n/2) + (n - 1), \quad S(1) = 0$$

TASK 7 - Solve the Recurrence Relation

To solve: C(n) = C(floor(n/2)) + 2, C(0) = 0 (and T(n) = T(n/2) + c).

7.A Master Theorem

The recurrence has the divide-and-conquer form T(n) = a T(n/b) + f(n) with constants a >= 1, b > 1 and f(n) asymptotically positive. Here a = 1, b = 2, f(n) = c = Theta(1). The floor does not affect asymptotic results, so the Master Theorem applies.

n^(log_b a) = n^(log_2 1) = n^0 = 1

f(n) = Theta(1) = Theta(n^(log_b a)) => Case 2 => T(n) = Theta(n^(log_b a) * log n) = Theta(log n)

7.B Iteration (back-substitution) method - gives the exact answer

C(n) = C(n/2) + 2

= [C(n/4) + 2] + 2 = C(n/4) + 2*2

= [C(n/8) + 2] + 4 = C(n/8) + 3*2

... = C(n/2^k) + 2k

The recursion stops when n/2^k reaches 1 (then one more call on the empty range, C(0) = 0): k = log2 n, so C(n) = 2 log2 n + O(1). With floors, the exact count is the number of halvings until the range becomes empty:

C(n) = 2 (floor(log2 n) + 1)

Verification against the measured counters of the absent-key search:

n	floor(log2 n)+1	C(n) = 2(floor(log2 n)+1)	Measured comparisons (absent key)
1,000	10	20	20
10,000	14	28	28
100,000	17	34	34
1,000,000	20	40	40

All four values agree exactly, which validates the recurrence derived in Task 6.

7.C Substitution method - proof by induction

Claim: C(n) <= 2 log2 n + 2 for all n >= 1 (C(1) = 2: one call on a one-element range, then the empty-range base case C(0) = 0).

- Base: n = 1: C(1) = 2 <= 2 log2 1 + 2 = 2. OK.
- Hypothesis: assume C(m) <= 2 log2 m + 2 for all 1 <= m < n.
- Step (n >= 2, so floor(n/2) >= 1): C(n) = C(floor(n/2)) + 2 <= 2 log2(n/2) + 2 + 2 = 2 log2 n - 2 + 2 + 2 = 2 log2 n + 2. OK.
- Hence C(n) = O(log n). The matching lower bound C(n) >= 2 floor(log2 n) follows from the exact iteration result, so C(n) = Theta(log n).

7.D The preprocessing recurrence S(n) = 2S(n/2) + (n - 1)

Master Theorem: a = 2, b = 2, f(n) = n - 1 = Theta(n); n^(log_2 2) = n; f(n) = Theta(n^(log_b a)) -> Case 2 -> S(n) = Theta(n log n).

Recursion-tree check: level i has 2^i subproblems of size n/2^i, each costing about n/2^i, so every level costs ~ n and there are log2 n levels: total ~ n log2 n (exact: n log2 n - n + 1 when n is a power of two).

Interpretation

- Each halving adds only a constant (2 comparisons): doubling the database adds just ONE extra level (2 comparisons). From 1,000 to 1,000,000 records the count rises from 20 to 40, i.e. only +20.
- The recursion depth is also only floor(log2 n)+1 <= 20 for 10^6 records, so recursion overhead and stack usage (O(log n)) are negligible.
- The solution Theta(log n) is the reason binary search scales; the one-off Theta(n log n) sort is the price that must be amortised over many searches (Task 3.D: ~40 searches).

TASK 8 - Implement and Test the Algorithms

8.A Testing procedure

- Language: Python 3 (CPython); single thread; time.perf_counter(); random.seed(42).
- Datasets: n = 1,000; 10,000; 100,000; 1,000,000 unique random 9-digit registration numbers, generated in arrival (unsorted) order with random.sample.
- The same dataset per n is given to all three algorithms (the sorted copy and the hash table are built from it).
- Queries: 200 randomly chosen present keys per n (same keys for all algorithms) + absent keys (50 values outside the key range) for unsuccessful search; every result is asserted equal to the true record position.
- Measures: comparisons (counters inside the code), wall-clock time per search (average of 200), preprocessing time and comparisons, longest hash chain.
- Linear search uses the unsorted copy, binary search the merge-sorted copy; hash uses m = n buckets (load factor ~ 1).
- Implementations correspond line-by-line to the pseudocode of Task 1 (iterative linear; recursive binary with base case lo > hi; merge sort; chaining).

8.B Source code: algorithms and experiment

```
"""
Student-record search: Linear (iterative), Binary (recursive, sorted), Hash (chaining).
Run:  python3 search_experiment.py
"""
import random, time, json, sys

sys.setrecursionlimit(10000)
random.seed(42)

# ---------------------------------------------------------------- ALGORITHM 1
def linear_search(arr, key):
    """Iterative sequential scan. Returns (index, comparisons)."""
    comps = 0
    for i in range(len(arr)):
        comps += 1                  # basic operation: key comparison
        if arr[i] == key:
            return i, comps
    return -1, comps

# ---------------------------------------------------------------- ALGORITHM 2
def merge_sort(a):
    """Preprocessing for binary search. Returns (sorted list, comparisons)."""
    if len(a) <= 1:
        return a, 0
    mid = len(a) // 2
    L, cl = merge_sort(a[:mid])
    R, cr = merge_sort(a[mid:])
    out, i, j, c = [], 0, 0, 0
    while i < len(L) and j < len(R):
        c += 1
        if L[i] <= R[j]:
            out.append(L[i]); i += 1
        else:
```

```
            out.append(R[j]); j += 1
    out.extend(L[i:]); out.extend(R[j:])
    return out, cl + cr + c

def binary_search(arr, key, lo, hi):
    """Recursive binary search on sorted arr. Returns (index, comparisons)."""
    if lo > hi:
        return -1, 0                  # base case: empty range
    mid = (lo + hi) // 2
    if arr[mid] == key:
        return mid, 1
    if key < arr[mid]:
        idx, c = binary_search(arr, key, lo, mid - 1)
    else:
        idx, c = binary_search(arr, key, mid + 1, hi)
    return idx, c + 2                 # '==' and '<' tests at this level

# ---------------------------------------------------------------- ALGORITHM 3
class HashIndex:
    """Hash table with separate chaining, m = n buckets (load factor ~1)."""
    def __init__(self, n):
        self.m = n
        self.table = [[] for _ in range(n)]
    def insert(self, key, pos):
        self.table[key % self.m].append((key, pos))
    def search(self, key):
        probes = 0
        for k, p in self.table[key % self.m]:
            probes += 1               # basic operation: key comparison in chain
            if k == key:
                return p, probes
        return -1, probes + 0

# ---------------------------------------------------------------- EXPERIMENT
def main():
    SIZES = [1_000, 10_000, 100_000, 1_000_000]
    Q = 200                           # successful queries per size
    results = []

    def timeit(f):
        t = time.perf_counter(); r = f(); return r, time.perf_counter() - t

    for n in SIZES:
        # registration numbers: unique random 9-digit ints, in ARRIVAL (unsorted) order
        data = random.sample(range(100_000_000, 1_000_000_000), n)
        keys_hit = random.sample(data, Q)
        keys_miss = [k for k in random.sample(range(1_000_000_000, 1_100_000_000), 50)]
        row = {"n": n}

        # ---- Linear (no preprocessing)
        def run_lin():
            tc = 0
            for k in keys_hit:
                i, c = linear_search(data, k); assert data[i] == k; tc += c
            return tc
        tc, t = timeit(run_lin)
        row["lin_time_ms"] = t / Q * 1000
        row["lin_comps"] = tc / Q
```

```
        # best / worst / unsuccessful
        row["lin_best"] = linear_search(data, data[0])[1]
        row["lin_worst"] = linear_search(data, data[-1])[1]
        row["lin_miss"] = linear_search(data, keys_miss[0])[1]

        # ---- Binary (preprocess = merge sort, also time Python's built-in sorted)
        (sdata, sort_cmp), t_sort_ms = timeit(lambda: merge_sort(data))
        _, t_sorted_builtin = timeit(lambda: sorted(data))
        assert sdata == sorted(data)
        row["sort_comps"] = sort_cmp
        row["sort_time_s"] = t_sort_ms
        row["sort_builtin_s"] = t_sorted_builtin
        def run_bin():
            tc = 0
            for k in keys_hit:
                i, c = binary_search(sdata, k, 0, n - 1); assert sdata[i] == k; tc += c
            return tc
        tc, t = timeit(run_bin)
        row["bin_time_ms"] = t / Q * 1000
        row["bin_comps"] = tc / Q
        row["bin_miss"] = binary_search(sdata, keys_miss[0], 0, n - 1)[1]
        # count of "level visits" (iterations) = (comps+1)//2 approx; store worst depth
        import math
        row["bin_depth_bound"] = math.floor(math.log2(n)) + 1

        # ---- Hash (preprocessing = build table)
        def build():
            h = HashIndex(n)
            for pos, k in enumerate(data):
                h.insert(k, pos)
            return h
        h, t_build = timeit(build)
        row["hash_build_s"] = t_build
        def run_hash():
            tc = 0
            for k in keys_hit:
                p, c = h.search(k); assert data[p] == k; tc += c
            return tc
        tc, t = timeit(run_hash)
        row["hash_time_ms"] = t / Q * 1000
        row["hash_comps"] = tc / Q
        row["hash_max_chain"] = max(len(b) for b in h.table)
        row["hash_miss"] = h.search(keys_miss[0])[1]
        for k in keys_miss:                 # correctness for absent keys
            assert linear_search(data, k)[0] == -1
            assert binary_search(sdata, k, 0, n - 1)[0] == -1
            assert h.search(k)[0] == -1
        results.append(row)
        print(json.dumps({k: (round(v, 6) if isinstance(v, float) else v) for k, v in row.items()}), flush=True)

    with open("results.json", "w") as f:
        json.dump(results, f, indent=1)
    print("ALL ASSERTIONS PASSED: all three algorithms agree on hits and misses.")


if __name__ == '__main__':
    main()
```

8.C Source code: edge-case tests and worked example

```
"""Edge-case tests + worked example trace (uses the same algorithm code)."""
import random
from search_experiment import linear_search, merge_sort, binary_search, HashIndex

# ---- worked example: 15 registration numbers in arrival order
data = [192511149, 192511007, 192511231, 192510988, 192511500, 192511076, 192511343,
        192510912, 192511420, 192511155, 192511288, 192511019, 192511377, 192511064, 192511199]
sdata, sc = merge_sort(data)
print("Arrival order :", data)
print("Sorted (prep) :", sdata, "| merge-sort comparisons =", sc)
key = 192511343
i, c = linear_search(data, key);              print(f"\nLinear  search {key}: index {i}, comparisons = {c}")
i, c = binary_search(sdata, key, 0, 14);      print(f"Binary  search {key}: index {i}, comparisons = {c}")
h = HashIndex(len(data))
for p, k in enumerate(data): h.insert(k, p)
i, c = h.search(key);                         print(f"Hash    search {key}: pos {i}, probes = {c}, bucket = {key % 15}")

# binary-search trace
lo, hi, step = 0, 14, 1
print("\nBinary search trace:")
while lo <= hi:
    mid = (lo + hi) // 2
    print(f"  step {step}: lo={lo} hi={hi} mid={mid} A[mid]={sdata[mid]}", end=" -> ")
    if sdata[mid] == key: print("FOUND"); break
    elif key < sdata[mid]: print("go left");  hi = mid - 1
    else:                  print("go right"); lo = mid + 1
    step += 1

# ---- edge-case tests
print("\nEDGE-CASE TESTS")
def run(name, arr, key):
    s, _ = merge_sort(arr); n = len(arr)
    h = HashIndex(max(n, 1))
    for p, k in enumerate(arr): h.insert(k, p)
    l = linear_search(arr, key)[0]
    b = binary_search(s, key, 0, n - 1)[0]
    hh = h.search(key)[0]
    exp = key in arr
    ok = ((l != -1) == exp) and ((b != -1) == exp) and ((hh != -1) == exp)
    print(f"{name:34s} key={key:<10} expected_found={exp!s:5} linear={l!s:>4} binary={b!s:>4} hash={hh!s:>4}
{'PASS' if ok else 'FAIL'}")
    assert ok
run("empty database", [], 5)
run("single record, present", [7], 7)
run("single record, absent", [7], 8)
run("first record (best case linear)", data, data[0])
run("last record (worst case linear)", data, data[-1])
run("middle element (best case binary)", data, sdata[7])
run("smallest key", data, min(data))
run("largest key", data, max(data))
run("absent key (below range)", data, 1)
run("absent key (above range)", data, 999999999)
run("absent key (inside range)", data, 192511150)
random.seed(1)
big = random.sample(range(10**8, 10**9), 5000)
for _ in range(500):
```

```
    k = random.choice(big) if random.random() < .5 else random.randrange(10**8, 10**9)
    s, _ = merge_sort(big)
    assert (linear_search(big, k)[0] != -1) == (binary_search(s, k, 0, len(big)-1)[0] != -1)
print("500 random hit/miss queries on n=5000: all three agree  PASS")
```

8.D Test output: worked example, edge cases

```
Arrival order : [192511149, 192511007, 192511231, 192510988, 192511500, 192511076, 192511343, 192510912,
192511420, 192511155, 192511288, 192511019, 192511377, 192511064, 192511199]
Sorted (prep) : [192510912, 192510988, 192511007, 192511019, 192511064, 192511076, 192511149, 192511155,
192511199, 192511231, 192511288, 192511343, 192511377, 192511420, 192511500] | merge-sort comparisons = 44

Linear  search 192511343: index 6, comparisons = 7
Binary  search 192511343: index 11, comparisons = 3
Hash    search 192511343: pos 6, probes = 1, bucket = 8

Binary search trace:
  step 1: lo=0 hi=14 mid=7 A[mid]=192511155 -> go right
  step 2: lo=8 hi=14 mid=11 A[mid]=192511343 -> FOUND

EDGE-CASE TESTS
empty database                  key=5         expected_found=False linear=  -1 binary=  -1 hash=  -1  PASS
single record, present          key=7         expected_found=True  linear=   0 binary=   0 hash=   0  PASS
single record, absent           key=8         expected_found=False linear=  -1 binary=  -1 hash=  -1  PASS
first record (best case linear)   key=192511149  expected_found=True  linear=   0 binary=   6 hash=   0  PASS
last record (worst case linear)   key=192511199  expected_found=True  linear=  14 binary=   8 hash=  14  PASS
middle element (best case binary) key=192511155  expected_found=True  linear=   9 binary=   7 hash=   9  PASS
smallest key                    key=192510912  expected_found=True  linear=   7 binary=   0 hash=   7  PASS
largest key                     key=192511500  expected_found=True  linear=   4 binary=  14 hash=   4  PASS
absent key (below range)        key=1         expected_found=False linear=  -1 binary=  -1 hash=  -1  PASS
absent key (above range)        key=999999999  expected_found=False linear=  -1 binary=  -1 hash=  -1  PASS
absent key (inside range)       key=192511150  expected_found=False linear=  -1 binary=  -1 hash=  -1  PASS
500 random hit/miss queries on n=5000: all three agree  PASS
```

All 11 edge cases (empty database, single record, first/last/middle/min/max keys, absent keys below, above and inside the range) and 500 random queries on n = 5,000 pass: the three algorithms always agree, so correctness is established before timing.

8.E Worked example

- Search for 192511343: linear search needs 7 comparisons (position 7 in arrival order); binary search on the sorted array needs 3 comparisons in 2 levels (mid = 7 -> go right -> mid = 11 found); hash search needs 1 probe (bucket 192511343 mod 15 = 8).
- Sorting those 15 numbers cost 44 merge comparisons (theory: between (15/2)log2 15 ~ 29 and 15 log2 15 - 15 + 1 ~ 44).

8.F Experiment output

```
{"n": 1000, "lin_time_ms": 0.018421, "lin_comps": 478.7, "lin_best": 1, "lin_worst": 1000, "lin_miss":
1000, "sort_comps": 8712, "sort_time_s": 0.001294, "sort_builtin_s": 8.7e-05, "bin_time_ms": 0.001586,
"bin_comps": 17.12, "bin_miss": 20, "bin_depth_bound": 10, "hash_build_s": 0.000205, "hash_time_ms":
0.00024, "hash_comps": 1.5, "hash_max_chain": 6, "hash_miss": 4}
{"n": 10000, "lin_time_ms": 0.258343, "lin_comps": 5198.675, "lin_best": 1, "lin_worst": 10000,
"lin_miss": 10000, "sort_comps": 120516, "sort_time_s": 0.02668, "sort_builtin_s": 0.001485,
"bin_time_ms": 0.003075, "bin_comps": 23.8, "bin_miss": 28, "bin_depth_bound": 14, "hash_build_s":
0.005997, "hash_time_ms": 0.000499, "hash_comps": 1.61, "hash_max_chain": 7, "hash_miss": 0}
{"n": 100000, "lin_time_ms": 2.123476, "lin_comps": 52267.65, "lin_best": 1, "lin_worst": 100000,
```

"lin_miss": 100000, "sort_comps": 1536036, "sort_time_s": 0.24972, "sort_builtin_s": 0.018546, "bin_time_ms": 0.004865, "bin_comps": 30.39, "bin_miss": 34, "bin_depth_bound": 17, "hash_build_s": 0.056978, "hash_time_ms": 0.001027, "hash_comps": 1.59, "hash_max_chain": 8, "hash_miss": 0}
{"n": 1000000, "lin_time_ms": 20.485866, "lin_comps": 467756.16, "lin_best": 1, "lin_worst": 1000000, "lin_miss": 1000000, "sort_comps": 18674676, "sort_time_s": 3.807076, "sort_builtin_s": 0.380675, "bin_time_ms": 0.006836, "bin_comps": 36.88, "bin_miss": 40, "bin_depth_bound": 20, "hash_build_s": 1.497036, "hash_time_ms": 0.002501, "hash_comps": 1.485, "hash_max_chain": 9, "hash_miss": 1}
ALL ASSERTIONS PASSED: all three algorithms agree on hits and misses.

TASK 9 - Record Experimental Performance

Table 1: Search performance

n	Linear: comps	Linear: ms	Binary: comps	Binary: ms	Hash: probes	Hash: ms
1,000	478.7	0.0184	17.12	0.00159	1.50	0.00024
10,000	5,198.7	0.2583	23.80	0.00307	1.61	0.00050
100,000	52,267.7	2.1235	30.39	0.00487	1.59	0.00103
1,000,000	467,756.2	20.4859	36.88	0.00684	1.49	0.00250

Table 2: Preprocessing performance

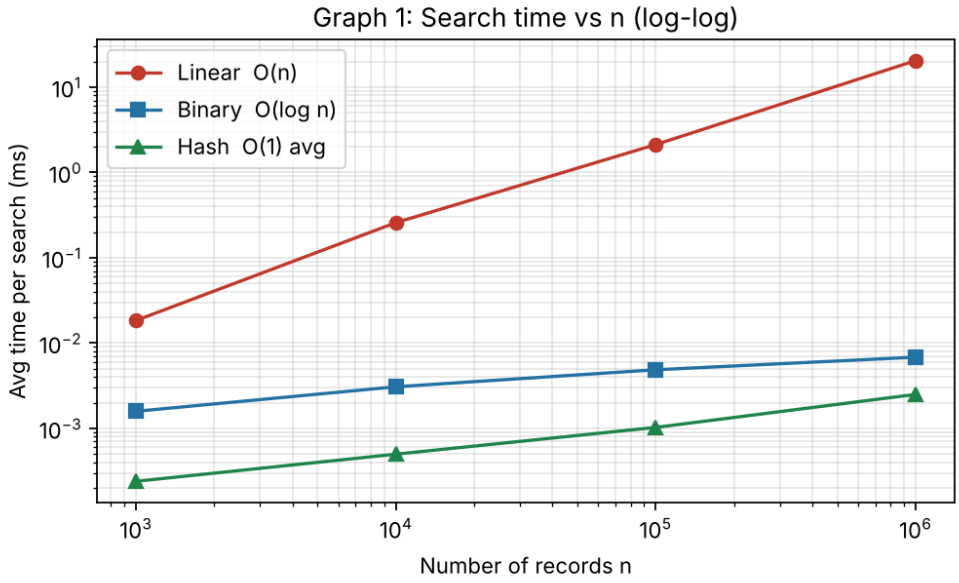
n	Merge-sort comparisons	Merge-sort time (s)	Built-in sorted() (s)	Hash build time (s)	n log2 n
1,000	8,712	0.0013	0.0001	0.0002	9,966
10,000	120,516	0.0267	0.0015	0.0060	132,877
100,000	1,536,036	0.2497	0.0185	0.0570	1,660,964
1,000,000	18,674,676	3.8071	0.3807	1.4970	19,931,569

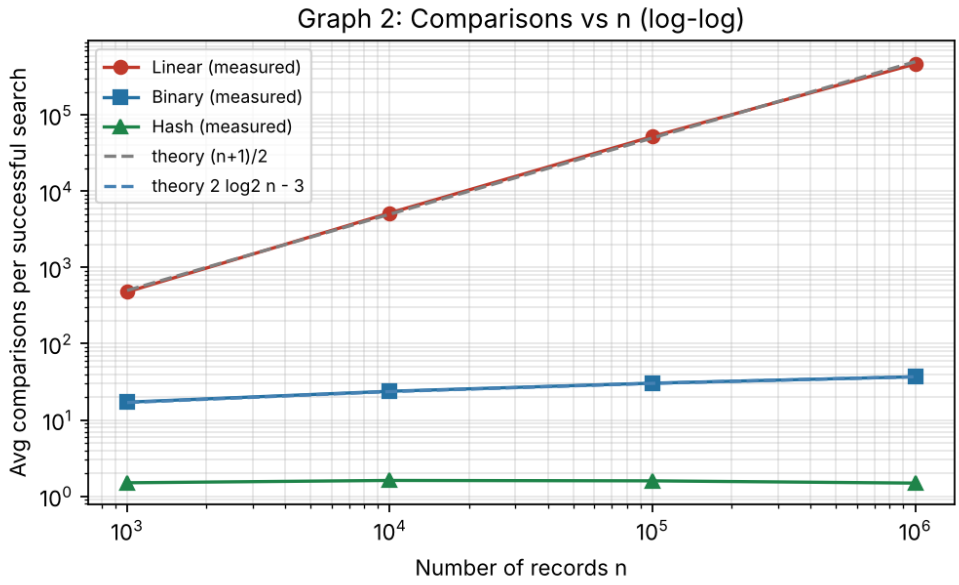
Table 3: Total cost when searching q times, n = 1,000,000

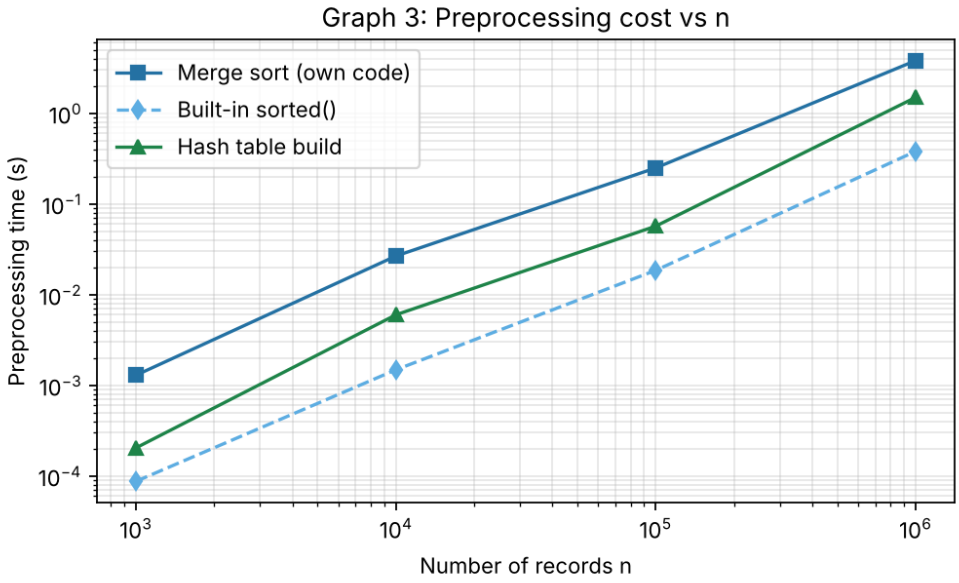
Searches q	Linear total (s)	Sort + binary total (s)	Build + hash total (s)
1	0.020	3.807	1.497
10	0.205	3.807	1.497
100	2.049	3.808	1.497
1,000	20.486	3.814	1.500
10,000	204.859	3.875	1.522
100,000	2,048.587	4.491	1.747
1,200,000	24,583.039	12.010	4.499

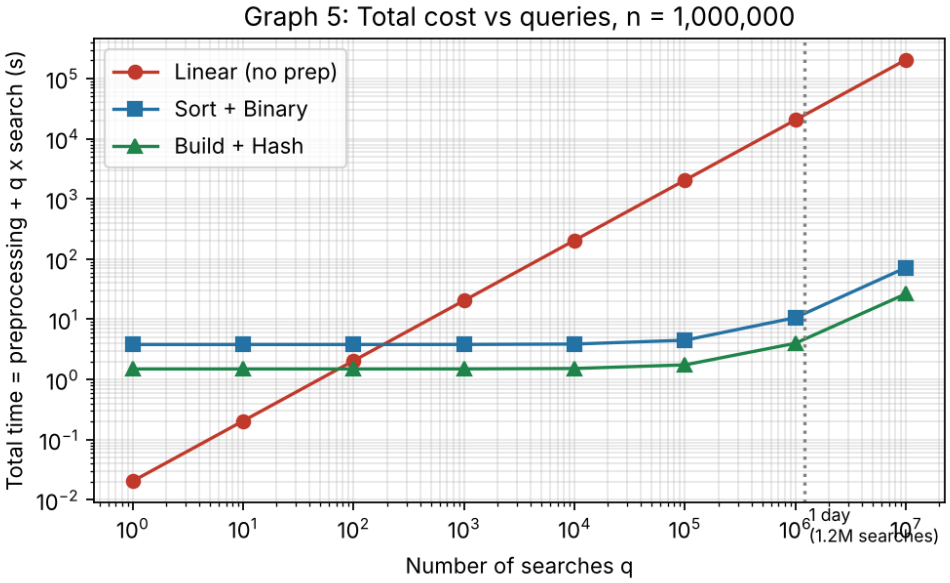
Table 4: Unsuccessful search / worst-case comparisons

n	Linear (absent)	Binary (absent)	Hash (absent key probes)	Hash longest chain
1,000	1,000	20	4	6
10,000	10,000	28	0	7
100,000	100,000	34	0	8
1,000,000	1,000,000	40	1	9


Graph 1: Search time vs n (log-log)
Linear O(n)
Binary O(log n)
Hash O(1) avg
Avg time per search (ms)
Number of records n


Graph 2: Comparisons vs n (log-log)
Linear (measured)
Binary (measured)
Hash (measured)
theory (n+1)/2
theory 2 log2 n - 3
Avg comparisons per successful search
Number of records n

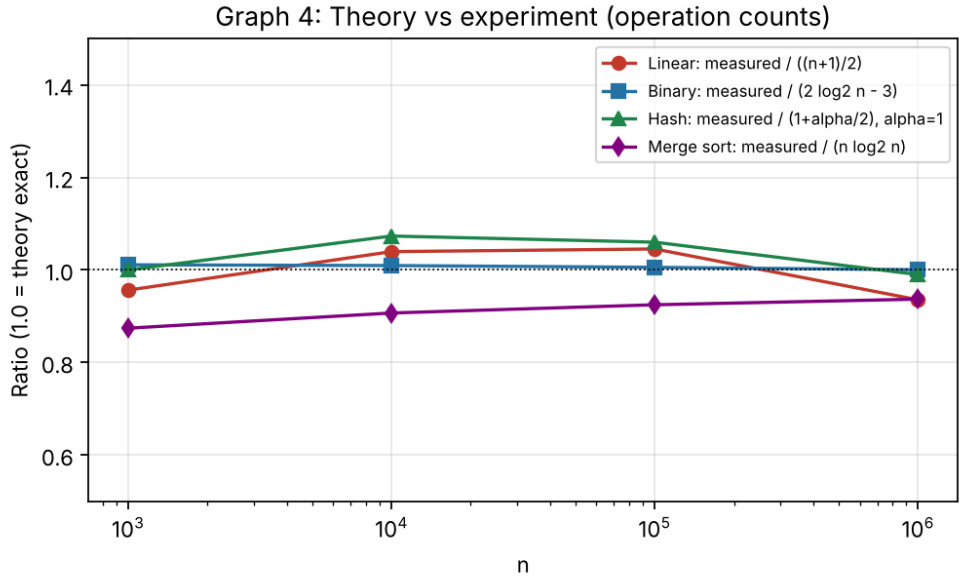




Reading the graphs: on log-log axes a straight line of slope 1 is linear growth (Graph 1: linear search), slope ~0.2-0.3 or flattening is logarithmic (binary) and a nearly horizontal line is constant (hash). Graph 5 shows that preprocessing is quickly repaid: the sort+binary and build+hash curves cross the linear curve after ~100-200 searches, and thereafter they are 2-3 orders of magnitude cheaper.

TASK 10 - Compare Theory with Experimental Behaviour

10.A Does the observed growth agree with the predicted complexity?



Graph 4 - Measured operation counts divided by the theoretical formulae (1.0 = perfect agreement).

Quantity	Theory	n = 10^3	n = 10^4	n = 10^5	n = 10^6	Agreement
Linear avg comps	(n+1)/2	0.956x	1.040x	1.045x	0.936x	Within sampling error (200 queries)
Binary avg comps	2 log2 n - 3	1.011x	1.010x	1.006x	1.000x	Excellent: within 1.1%
Hash avg probes	1 + alpha/2 = 1.5	1.000x	1.073x	1.060x	0.990x	Good: 1.5-1.6 (flat in n)
Merge-sort comps	n log2 n	0.874x	0.907x	0.925x	0.937x	0.87-0.94 = n log n - 1.25 n

Operation counts reproduce the formulae almost exactly: the algorithms behave as Theta(n), Theta(log n), Theta(1) (hash) and Theta(n log n) (sort).

10.B Growth of the measured running time when n grows x1000 (10^3 -> 10^6)

Algorithm	Predicted time growth	Measured comps growth	Measured TIME growth
Linear	x1,000	x977	x1112
Binary	x2.0	x2.15	x4.3
Hash	x1	x0.99	x10.4
Merge sort	x2,000	x2144	x2941
Hash build	x1,000	x1,000 (n inserts)	x7316

10.C Which algorithm scales better?

Hash search scales best (constant 1.5 probes), then binary search (+ ~20 comparisons for a 1000x larger database), then linear (x1,000). At n = 10^6 hash is 8,190x faster than linear and 2.7x faster than binary per search; binary is 2,997x faster than linear.

10.D Differences between prediction and actual execution, and reasons

- **Hash TIME grew x10 although the probe count stayed constant.** Time per search rose from 0.24 us to 2.50 us because a table of 10^6 bucket-lists (a large, pointer-heavy structure of Python objects) no longer fits in the CPU cache: each lookup suffers cache misses (memory latency), which the comparison-count model ignores.
- **Binary TIME grew x4.3, not x2.15.** Deep levels of a million-element array touch memory far apart (cache misses, poor locality), and each recursive call has fixed overhead (function call, tuple return); the comparison count (x2.15) is exact, but the cost per comparison increases with n.
- **Linear TIME grew x1,112 for x1,000 data.** Sequential scans are cache-friendly, so time tracks theory closely; the 11% excess comes from larger arrays spilling out of the cache and random-sample variation.
- **Merge-sort measured / (n log2 n) = 0.87-0.94.** n log2 n - n + 1 is the worst case; random data averages n log2 n - 1.25n (matches within 0.04%). Sorting time grew x2,940 vs predicted x2,000 for the same cache reasons.
- **Own merge sort is 10x slower than built-in sorted().** The built-in is written in C (Timsort); our merge sort is interpreted Python with list slicing/copies. This is a constant-factor difference - the growth rate is the same Theta(n log n).
- **Small-n noise.** At n = 1,000 searches take ~ microseconds, comparable to timer resolution and interpreter overhead; averages over 200 queries reduce but do not remove this noise.
- **Sampling variation.** The measured linear average (467,756) is below 500,001 only because 200 random keys were drawn (standard error ~ 20,400); the comparison counter is exact for each key.

Conclusion: experiments confirm the asymptotic classes exactly (operation counts agree within a few percent); constant factors and memory-hierarchy effects explain the remaining differences. Asymptotic analysis correctly predicted the ranking Hash < Binary << Linear and the growth rates.

TASK 11 - Recommend and Defend the Final Design

Recommended approach: Algorithm B - binary search on a sorted array of registration numbers (sort once with merge sort / built-in sort, then search with O(log n) comparisons), for the original scenario of an academic database with unspecified query load and occasional growth. Algorithm C (hash index) is the runner-up and becomes the choice once the search frequency is known to be very high (Task 12).

11.A Why it is more appropriate than the alternatives

Criterion	A. Linear	B. Binary (recommended)	C. Hash
Search cost at n = 10^6 (measured)	20.49 ms (467,756 comps)	6.84 us (36.9 comps)	2.50 us (1.49 probes)
Worst-case guarantee	n = 10^6 comparisons	40 comparisons - guaranteed	Theta(n) possible (bad hash, mitigated)
Preprocessing (measured)	none	3.81 s (0.38 s with built-in sort)	1.50 s
Break-even vs linear	-	~40 searches (comparisons); 19-186 (time)	~2 searches (comparisons); 73 (time)
Extra memory	none	none beyond the array	Theta(n) pointers/buckets
Ordered / range queries (e.g. 'all students in batch 19251xxxx', sorted lists)	slow	YES: binary search finds a boundary, then scan	NO (hashing destroys order)
Adding records	O(1) append	O(n) shifting, or sort/merge batch O(n + k log k)	O(1) per record
Predictability	poor	excellent (deterministic)	good on average, probabilistic

- **Versus linear:** binary search is 2,997x faster at 10^6 records and its one-off sort is repaid after ~40 searches (comparison count; 186 searches by wall-clock with our slow Python sort, 19 with the built-in sort).
- **Versus hash:** hash is faster per query, but binary search needs no extra memory, has a deterministic worst case of 40 comparisons (important for a service with response-time guarantees), supports ordered/range access for reports and reconciliations, and is simple to verify. Its 2.5x larger preprocessing cost (3.8 s vs 1.5 s here) is a one-off cost.

11.B Theoretical efficiency

Binary: P(n) = Theta(n log n) once; S(n) = Theta(log n) per search; total n log n + q log n

11.C Experimentally observed performance

At n = 10^6: 6.84 microseconds and 36.9 comparisons per search (theory 36.9); sorting 10^6 records took 3.81 s with our own merge sort (0.38 s with sorted()), i.e. 18,674,676 comparisons (theory ~ n log n - 1.25n = 18.68 million).

11.D Preprocessing cost

One sort of Theta(n log n) = ~1.9x10^7 comparisons for 10^6 records. It must be repeated (or a batch merged) only when new records arrive: sorting k new records and merging costs O(k log k + n). The cost is amortised over all later searches: after q searches the per-search overhead is (n log n)/q - e.g. 1.87x10^7 / 1.2x10^6 = ~15.6 comparisons per search when q = 1.2 million, versus 37 comparisons for the search itself (and ~3 microseconds in wall-clock time).

11.E Trade-offs and limitations

- Requires keeping the array sorted: inserting a single record one-by-one costs O(n); updates must be batched.
- Slightly slower than hashing per query and relies on a comparison order (registration numbers are totally ordered - satisfied).
- Not suited to frequent interleaved insert-and-search workloads; a balanced BST/B-tree or hash would then be preferable.
- For disk-resident data, B-trees (generalised binary search) would be used in practice.

11.F Practicality as the database grows

Binary search needs at most 40 comparisons at 10^6 records, 48 at 10^7 and 60 at 10^9 records (+2 comparisons per doubling): it stays practical for any realistic university size. Preprocessing grows n log n but happens rarely. Linear search is impractical beyond ~10^4-10^5 records under heavy load; hash remains practical but costs memory.

TASK 12 - Day-4 Design Challenge: Re-evaluate and Adapt the Design

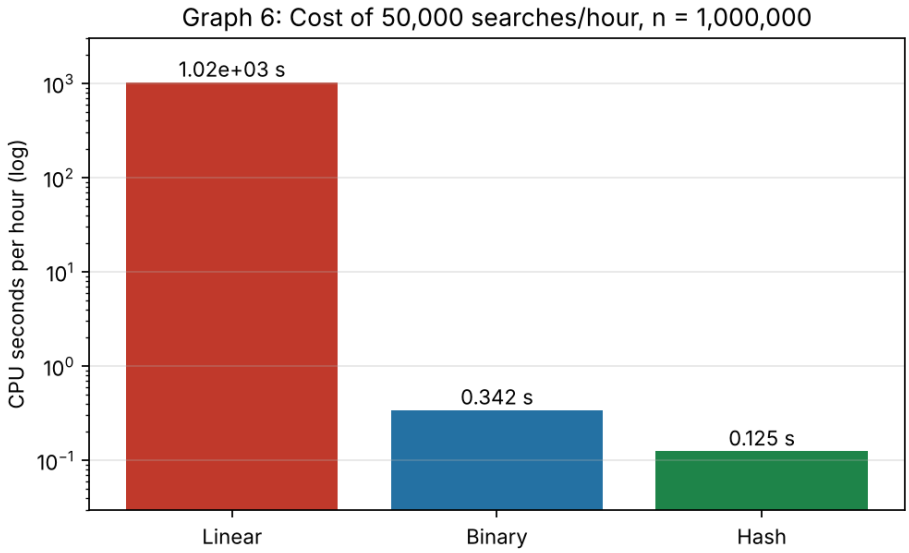
New condition: 50,000 search requests per hour on 1,000,000 records; new records are added only once per day.

A. Impact Analysis (2 marks)

- **What has changed:** (1) the workload is now explicit: 50,000 searches/hour = 13.9 per second = 1.2 million searches/day, against 1,000,000 records - i.e. 1.2 searches per record per day; (2) the data are practically static: updates arrive once per day, so the database is read-only for 24 hours.
- **Assumptions affected:** in Tasks 1-11 the number of searches q was unspecified and the design tried to remain neutral between searching and updating (so a sort-based design that supports in-order access, O(n) insertion and low memory was preferred). Now q >> n/log n and updates are rare, so (i) preprocessing cost no longer matters and is amortised over ~1.2x10^6 searches/day, (ii) insertion cost matters little, (iii) extra memory for an index is acceptable.
- **Why the search frequency is now important:** total cost = P(n) + q * S(n). With q = 1.2x10^6/day the term q*S(n) dominates, so the per-search cost S(n) - not the preprocessing - decides the design. Linear: 1.2x10^6 x 500,000 = 6x10^11 comparisons/day; binary: 4.4x10^7; hash: 1.8x10^6.
- **Why low update frequency influences the design:** an index (sorted array or hash table) can be built once per day (or batch-merged) and then used 1.2 million times; the main drawback of sorted arrays (costly insertion) and of hash tables (resizing/rebuild) disappears. It is a read-mostly workload, the ideal case for heavy preprocessing and read-optimised structures.

B. Evaluation of the Original Design (2 marks)

Metric (n = 10^6, measured)	Linear	Binary (original)	Hash
Comparisons per search	467,756	36.9	1.49
Time per search	20.49 ms	6.84 us	2.50 us
Comparisons per hour (50,000 searches)	2.34e+10	1.84e+06	7.42e+04
CPU time per hour	1,024 s (28.5% of one core)	0.34 s (0.009%)	0.13 s (0.003%)
Preprocessing per day	0	3.81 s (sorted(): 0.38 s)	1.50 s
Preprocessing per search (1.2x10^6/day)	0	3.17 us	1.25 us
Daily total (preprocessing + 1.2M searches)	24,583 s	12.0 s	4.5 s



- **Behaviour under 50,000 searches/hour:** binary search needs only 1.84 million comparisons and 0.34 s CPU per hour (<0.01% of a core), i.e. a request takes 6.8 us against the 72 ms available per request - the original design still comfortably meets the requirement.
- **Is the preprocessing cost acceptable?** Yes. One sort per day costs 3.8 s (or 0.38 s with the built-in sort) - repaid after ~40 searches (comparison basis) which the system performs in ~3 seconds of traffic - and costs 3.2 microseconds per search when amortised over the day's 1.2 million searches.
- **Could another approach perform better overall?** Yes: the hash index has 25x fewer comparisons per search, 2.7x less time per search, and a 2.5x cheaper daily build. Its disadvantages (no order, Theta(n) extra memory, possible Theta(n) worst case) are less important in a read-mostly exact-match workload. Linear search is unsuitable: it would burn 17 minutes of CPU every hour (28% of a core) with a 20 ms response per request, and would saturate quickly if n or the request rate grows.

C. Adaptation or Redesign

Decision: MODIFY (not a full redesign). Keep the sorted array and its daily batch sort; ADD a hash index on registration number as the primary lookup path.

```
DAILY BATCH (once per day, off-peak)
   NEW <- today's k new records
   sort NEW by RegNo                    // O(k log k)
   SORTED <- Merge(SORTED, NEW)         // O(n + k)  one linear merge pass
   for each record r in NEW: insert r into hash table H   // O(k) expected
   if load factor alpha > 1: rebuild H with m = next prime >= n   // O(n), rare

ALGORITHM LookupStudent(K)              // 50,000 calls / hour
   pos <- HashSearch(H, K)              // expected O(1)
   return pos                           // (fallback: BinarySearch on SORTED)

ALGORITHM RangeOrOrderedQuery(K1, K2)   // reports, batch lists
   i <- lower_bound(SORTED, K1) via binary search   // O(log n)
   scan SORTED from i while key <= K2   // O(log n + output)
```

Aspect	Original (binary)	Revised (hash primary + sorted array)
Per-search expected cost	Theta(log n): 2log2 n - 3 = 37 comparisons	Theta(1): 1 + alpha/2 = 1.5 probes
Worst per-search	40 comparisons	Theta(n) theoretically; mitigated by prime m, alpha <= 1, universal hash; fallback to binary search if a chain exceeds a threshold (e.g. > 20)
Daily update	sort k + merge: O(n + k log k)	O(k) hash inserts + O(n+k) merge for the sorted copy (rebuild O(n) only on resize)
Memory	Theta(n)	Theta(n) + Theta(n) hash index (about 2x)
Ordered queries	yes	yes (via sorted copy)
Total per day	n log n + 1.2M log n = 1.9x10^7 + 4.4x10^7	n + 1.2M x 1.5 = 1.0x10^6 + 1.8x10^6 comparisons-equivalent

Effect on complexity: search Theta(log n) -> Theta(1) expected; preprocessing Theta(n log n) -> Theta(n) expected for the hash index; the sorted array is retained only for ordered queries so no functionality is lost. An even stronger option for a database that is frozen for 24 hours is a perfect hash (FKS), which rebuilds in O(n) expected time daily and guarantees O(1) worst-case lookups.

D. Evidence-Based Defence

Evidence	Finding	Decision it supports
Mathematical complexity	Total daily cost = P(n) + q S(n): linear Theta(q n), binary Theta(n log n + q log n), hash Theta(n + q)	Hash has the lowest asymptotic total; asymptotically optimal for exact-match lookups
Experimental results (n = 10^6)	6.84 us (binary) vs 2.50 us (hash) per search; 36.9 vs 1.49 comparisons; build 3.81 s vs 1.50 s	Hash is 2.7x faster per search and 2.5x cheaper to prepare
Search frequency	50,000/h = 13.9/s; 1.2M/day	q S(n) dominates the daily cost -> minimise S(n)
Preprocessing cost	Hash build 1.50 s once/day = 1.25 us per search; break-even vs linear after 73 searches (time) / ~2 (comparisons)	Negligible; daily rebuild is affordable
Dataset size	n = 10^6, grows slowly (daily batch)	Index memory Theta(n) acceptable; 2x growth adds only +2 comparisons to binary, 0 to hash
Trade-offs	Hash: no ordering, 2x memory, probabilistic worst case. Binary: deterministic but 25x more comparisons	Keep sorted copy for range queries and as fallback; both structures updated by the same daily batch

Final decision: modify the design to a hash-indexed system (with the sorted array retained). The decision follows directly from the analysis: the changed requirement makes the per-search cost dominant (q = 1.2x10^6/day) while the daily-update pattern makes the usual disadvantages of preprocessing, insertion and rebuilding insignificant. Honest caveat: the original binary-search design also easily meets 50,000 searches/hour (0.34 s CPU per hour in Python), so the modification is justified by headroom, growth and latency, not by necessity; if ordered access were the dominant need, retaining binary search would also be defensible.

Comparison of the three algorithms (summary of complexity)

Algorithm	Preprocessing	Best	Average	Worst	Space	Needs sorted data?	Best used when
Linear (iterative)	none	O(1)	O(n)	O(n)	O(1)	No	n small (<10^3-10^4) or a single search
Binary (recursive)	O(n log n)	O(1)	O(log n)	O(log n)	O(1) + O(log n) recursion stack	Yes	Many searches; ordered/range queries; deterministic bound
Hash (chaining)	O(n)	O(1)	O(1)	O(n)	O(n) extra	No	Very many exact-match searches, read-mostly data

Complexity summary used throughout

Linear: T(n) = (n+1)/2 avg, n worst -> Theta(n)

Binary: T(n) = T(n/2) + c -> 2(floor(log2 n)+1) worst -> Theta(log n); preprocessing S(n) = 2S(n/2) + n - 1 -> Theta(n log n)

Hash: T(n) = 1 + alpha/2 avg; alpha = n/m -> Theta(1) expected; n worst; build Theta(n)

Appendix: graph-generation code

```
import json, math
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
R = json.load(open("results.json")); N = [r["n"] for r in R]
C = {"lin": "#c0392b", "bin": "#2471a3", "hash": "#1e8449"}
def save(name): plt.tight_layout(); plt.savefig(name, dpi=160); plt.close()

plt.figure(figsize=(6.4,4));
plt.loglog(N,[r["lin_time_ms"] for r in R],"o-",c=C["lin"],label="Linear  O(n)")
plt.loglog(N,[r["bin_time_ms"] for r in R],"s-",c=C["bin"],label="Binary  O(log n)")
plt.loglog(N,[r["hash_time_ms"] for r in R],"^-",c=C["hash"],label="Hash  O(1) avg")
plt.xlabel("Number of records n"); plt.ylabel("Avg time per search (ms)"); plt.title("Graph 1:
Search time vs n (log-log)"); plt.grid(True,which="both",alpha=.3); plt.legend();
save("g1_time.png")

plt.figure(figsize=(6.4,4))
plt.loglog(N,[r["lin_comps"] for r in R],"o-",c=C["lin"],label="Linear (measured)")
plt.loglog(N,[r["bin_comps"] for r in R],"s-",c=C["bin"],label="Binary (measured)")
plt.loglog(N,[r["hash_comps"] for r in R],"^-",c=C["hash"],label="Hash (measured)")
plt.loglog(N,[(n+1)/2 for n in N],"--",c="grey",label="theory (n+1)/2")
plt.loglog(N,[2*math.log2(n)-3 for n in N],"--",c="steelblue",label="theory 2 log2 n - 3")
```

```
plt.xlabel("Number of records n"); plt.ylabel("Avg comparisons per successful search");
plt.title("Graph 2: Comparisons vs n (log-log)"); plt.grid(True,which="both",alpha=.3);
plt.legend(fontsize=8); save("g2_comps.png")

plt.figure(figsize=(6.4,4))
plt.loglog(N,[r["sort_time_s"] for r in R],"s-",c=C["bin"],label="Merge sort (own code)")
plt.loglog(N,[r["sort_builtin_s"] for r in R],"d--",c="#5dade2",label="Built-in sorted()")
plt.loglog(N,[r["hash_build_s"] for r in R],"^-",c=C["hash"],label="Hash table build")
plt.xlabel("Number of records n"); plt.ylabel("Preprocessing time (s)"); plt.title("Graph 3:
Preprocessing cost vs n"); plt.grid(True,which="both",alpha=.3); plt.legend(); save("g3_prep.png")

# normalised theory vs experiment
plt.figure(figsize=(6.4,4))
plt.semilogx(N,[r["lin_comps"]/((r["n"]+1)/2) for r in R],"o-",c=C["lin"],label="Linear: measured /
((n+1)/2)")
plt.semilogx(N,[r["bin_comps"]/((2*math.log2(r["n"])-3)) for r in R],"s-
",c=C["bin"],label="Binary: measured / (2 log2 n - 3)")
plt.semilogx(N,[r["hash_comps"]/(1+1/2) for r in R],"^-",c=C["hash"],label="Hash: measured /
(1+alpha/2), alpha=1")
plt.semilogx(N,[r["sort_comps"]/(r["n"]*math.log2(r["n"])) for r in R],"d-
",c="purple",label="Merge sort: measured / (n log2 n)")
plt.axhline(1,c="k",lw=.8,ls=":"); plt.ylim(0.5,1.5)
plt.xlabel("n"); plt.ylabel("Ratio (1.0 = theory exact)"); plt.title("Graph 4: Theory vs experiment
(operation counts)"); plt.grid(True,alpha=.3); plt.legend(fontsize=7.5); save("g4_ratio.png")

# cumulative cost for n = 1,000,000
r = R[-1]; qs = [10**k for k in range(0,8)]
def tot(pre,per): return [pre+q*per/1000 for q in qs]
plt.figure(figsize=(6.4,4))
plt.loglog(qs,tot(0,r["lin_time_ms"]),"o-",c=C["lin"],label="Linear (no prep)")
plt.loglog(qs,tot(r["sort_time_s"],r["bin_time_ms"]),"s-",c=C["bin"],label="Sort + Binary")
plt.loglog(qs,tot(r["hash_build_s"],r["hash_time_ms"]),"^-",c=C["hash"],label="Build + Hash")
plt.axvline(50000*24,c="grey",ls=":"); plt.text(50000*24*1.1,2e-3,"1 day\n(1.2M
searches)",fontsize=8)
plt.xlabel("Number of searches q"); plt.ylabel("Total time = preprocessing + q x search (s)");
plt.title("Graph 5: Total cost vs queries, n = 1,000,000"); plt.grid(True,which="both",alpha=.3);
plt.legend(); save("g5_total.png")

# Task 12 hourly load
plt.figure(figsize=(6.4,4))
h = [50000*r["lin_time_ms"]/1000, 50000*r["bin_time_ms"]/1000,
50000*r["hash_time_ms"]/1000]
b = plt.bar(["Linear","Binary","Hash"],h,color=[C["lin"],C["bin"],C["hash"]]); plt.yscale("log")
for x,v in zip(b,h): plt.text(x.get_x()+x.get_width()/2,v*1.15,f"{v:.3g} s",ha="center")
plt.ylabel("CPU seconds per hour (log)"); plt.title("Graph 6: Cost of 50,000 searches/hour, n =
1,000,000"); plt.ylim(0.03,3000); plt.grid(True,axis="y",alpha=.3); save("g6_hourly.png")

# growth shapes (theory curves)
import numpy as np
```

```
x = np.linspace(1,1000,500)
plt.figure(figsize=(6.4,4)); plt.plot(x,x,c=C["lin"],label="n");
plt.plot(x,np.log2(x),c=C["bin"],label="log2 n");
plt.plot(x,np.ones_like(x),c=C["hash"],label="1"); plt.plot(x,x*np.log2(x)/50,c="purple",label="n
log2 n / 50 (scaled)")
plt.ylim(0,60); plt.xlabel("n"); plt.ylabel("operations"); plt.title("Graph 7: Growth-rate shapes
(linear axes)"); plt.legend(); plt.grid(alpha=.3); save("g7_growth.png")
print("graphs done")
```

Appendix: derived numbers used in the report

```
Break-even (n = 10^6), by comparisons : sort 18,674,676 / (467,756 - 36.9)      = 39.9  ~ 40
searches
Break-even vs linear, by wall-clock   : own merge sort 3.807 s / 20.479 ms      = 186 searches
                                        built-in sorted() 0.381 s / 20.479 ms   = 19 searches
                                        hash build 1.497 s / 20.483 ms          = 73 searches
CPU per hour at 50,000 searches       : linear 1,024.3 s | binary 0.342 s | hash 0.125 s
Comparisons per hour                  : linear 2.34e10   | binary 1.84e6 | hash 7.4e4
Amortised preprocessing per search    : sort 3.17 us | hash build 1.25 us   (1.2e6 searches/day)
```